IHTC Problem Breakdown

Maksym Byelko

June 2026

METAMODEL

Root / Container

Class: HospitalInstance
- Patients:[ ] ref– Patient
- Nurses:[ ] ref– Nurse
- Surgeons:[ ] ref– Surgeon
- Rooms:[ ] ref– Room
- Operating theatres:[ ] ref– OperatingTheatre
- decisionHorizon: int

Core Entities

Class: Patient
- id: int
- isMandatory: boolean (current workaround for matching based on Status)
- isScheduled: boolean (current workaround for comparison inside the predicate)
- dueDate: int
- releaseDate: int
- ageGroup: AgeGroup
- gender: Gender
- surgeryDuration: int
- stayLength: int
- incompatibleRooms:[ ] ref– Room
- assignedSurgeonId: ref– Surgeon

Class: Room
- id: int
- maxCapacity: int

Class: Nurse
- id: int
- skillLevel: int

Class: Surgeon
- id: int

Class: OperatingTheatre
- id: int

Temporal / Supporting Entities

Class: HospitalisationShift
- patientId: ref– Patient
- shift: int
- skillLevelRequired: int
- workload: int

Class: NurseRosterEntry
- nurseId: ref– Nurse
- shiftId: ref– HospitalisationShift
- maxWorkload: int

Class: SurgeonAvailability
- surgeonId: ref– Surgeon
- day: int
- maxOperatingTime: int

Class: OTAvailability
- OTid: ref– OperatingTheatre
- day: int
- maxCapacity: int

Class: RoomAvailability
- roomId: ref– Room
- day: int
- occupiedBeds: int

Class: DeletedAdmissionsTracker
- count: int

Decision / Assignment Entities

Class: Admission
- patientId: ref– Patient
- admissionDay: int
- roomId: ref– Room
- OTId: ref– OperatingTheatre
- isPostponed: ENUM YES/NO

Class: RoomShiftAssignment
- nurseId: ref– Nurse
- roomId: ref– Room
- shift: ref– HospitalisationShift

Boundary Data

Class: Occupant (subclass of Patient)
- patientId: ref– Patient
- assignedRoomId: ref– Room
- stayLength: int

PROBLEM

Decisions

D1: What day in the planning horizon will each patient be admitted?

D2: What operating theatre will each patient use?

D3: What recovery room will each patient use?

D4: What rooms will each nurse cover on their assigned shifts?

Model Transformations

0.1 Patient Admission and Removal

- Transformation 1: Insert a random non-mandatory patient with a random admission day, room, and theatre. Classes used: `Patient`, `Admission`, `Room`, `OperatingTheatre`. Purpose: to increase the number of scheduled patients and explore new possible allocations quickly. It is a broad exploratory move.

- Transformation 2: Insert the non-mandatory patient with the least workload into a feasible admission day, room, and theatre. Classes used: `Patient`, `Admission`, `Room`, `OperatingTheatre`, `SurgeonAvailability`, `OperatingTheatreAvailability`, `RoomAvailability`. Purpose: to add a patient who is easier to fit into the schedule.

- Transformation 3: Find the earliest available surgeon, then find a theatre and room available on the same day, and insert a random compatible non-mandatory patient. Classes used: `Patient`, `Admission`, `Surgeon`, `SurgeonAvailability`, `OperatingTheatre`, `OperatingTheatreAvailability`, `Room`, `RoomAvailability`. Purpose: to insert a patient in a way that is more strongly guided by actual resource availability, especially surgeon and theatre constraints.

- Transformation 4: Remove a random non-mandatory patient. Classes used: `Patient`, `Admission`. Purpose: to free up resources while only sacrificing an optional patient, which helps repair overloaded or poor-quality schedules.

- Transformation 5: Remove a random patient. Classes used: `Patient`, `Admission`. Purpose: to make a larger change to the schedule and free capacity, even if that means removing a more important patient.

- Transformation 6: Remove a random non-mandatory patient, then insert a random non-mandatory patient. Classes used: `Patient`, `Admission`, `Room`, `OperatingTheatre`, `SurgeonAvailability`, `OperatingTheatreAvailability`, `RoomAvailability`. Purpose: to swap one optional patient for another, testing whether a different patient gives a better schedule without changing the total number of optional patients.

- Transformation 7: Remove a random patient, then insert a random non-mandatory patient. Classes used: `Patient`, `Admission`, `Room`, `OperatingTheatre`, `SurgeonAvailability`, `OperatingTheatreAvailability`, `RoomAvailability`. Purpose: to replace part of the current schedule with a new optional patient, allowing a more disruptive change in the set of admitted patients.

0.2 Patient Reassignment

- Transformation 8: Give a random admitted patient a random new compatible room. Classes used: `Patient`, `Admission`, `Room`, `RoomAvailability`. Purpose: to reassign a patient to a different feasible room in order to improve room usage or reduce incompatibilities.

- Transformation 9: Give a random admitted patient a random new compatible admission day. Classes used: `Patient`, `Admission`, `SurgeonAvailability`, `OperatingTheatreAvailability`, `RoomAvailability`. Purpose: to move a patient to a different feasible day in order to improve schedule balance or resource usage.

- Transformation 10: Give a random admitted patient a random new compatible theatre. Classes used: `Patient`, `Admission`, `OperatingTheatre`, `OperatingTheatreAvailability`. Purpose: to reassign a patient to a different feasible theatre in order to improve theatre allocation.

- Transformation 11: Change a patient's room, admission day, and theatre sequentially. Classes used: `Patient`, `Admission`, `Room`, `RoomAvailability`, `OperatingTheatre`, `OperatingTheatreAvailability`, `SurgeonAvailability`. Purpose: to make a broader coordinated change to one patient's allocation across several decision variables.

- Transformation 12: Change a patient's room and admission day sequentially. Classes used: `Patient`, `Admission`, `Room`, `RoomAvailability`, `SurgeonAvailability`, `OperatingTheatreAvailability`. Purpose: to jointly adjust accommodation and timing in a way that may improve feasibility or quality more than changing one variable alone.

- Transformation 13: Change a patient's admission day and theatre sequentially. Classes used: `Patient`, `Admission`, `OperatingTheatre`, `OperatingTheatreAvailability`, `SurgeonAvailability`, `RoomAvailability`. Purpose: to jointly modify the timing and surgical allocation of a patient while keeping the room unchanged.

0.3 Nurse Assignment

- Transformation 14: Add a random nurse to a random room on a random compatible shift. Classes used: `Nurse`, `Room`, `Shift`, `Occupant`. Purpose: to create a new nurse-to-room assignment and explore alternative staffing allocations.

- Transformation 15: Remove a random nurse from a random room on their schedule. Classes used: `Nurse`, `Room`, `Shift`. Purpose: to free a nurse assignment and allow the optimisation to repair or rebalance staffing decisions.

- Transformation 16: Remove a nurse from one room and add them to another. Classes used: `Nurse`, `Room`, `Shift`, `Occupant`. Purpose: to reallocate nursing capacity between rooms without changing the total number of nurse assignments.

CONSTRAINTS

Patient Admission and Removal

- A patient must be assigned a feasible admission day.
- A patient must be assigned a feasible theatre, surgeon, and recovery room.
- The chosen surgeon must have capacity.
- The chosen operating theatre must have capacity.
- The chosen room must have capacity.
- For non-mandatory insertion moves, surgeon and nurse availability must exist.
- In transformation 3, the theatre and room must be available on the same day.
- In transformation 3, the inserted patient must be compatible with the chosen resources.
- Mandatory patients are prioritised in the initial feasible construction.
- Non-mandatory patients are not scheduled initially.
- Infeasible moves are permitted unless specified otherwise.
- Infeasible solutions receive a heavy penalty from the validator.

OBJECTIVES

Patients Admission and Removal

- Maximise the number of scheduled patients.
- Minimise surgeon capacity violations.
- Minimise theatre capacity violations.
- Minimise room capacity violations.
- Minimise Standard deviation of room occupancy.
- Minimise the number of removed patients.